

MANIPAL FORMAT | PRACTICE PAPER
THIRD SEMESTER B.TECH. | MIDSEM PRACTICE
DATA STRUCTURES [CSS 2101]
COMPUTER SCIENCE AND ENGINEERING
SET A: ENSEMBLE OF PAST-PAPER QUESTIONS

Marks: 30**Suggested duration: 90 minutes**

Answer all questions. Questions 1–5 are MCQs worth 1 mark each. Select one correct option per MCQ. The remaining questions are theory questions worth 25 marks in total; show working and draw labelled diagrams where appropriate. Modules 1-2; module 3 restricted to stacks. Independent practice paper; not an official university examination.

Section A: Multiple-choice questions (5 marks)

- 1) Which data structure directly supports undoing the most recent action first? (1)
(A) Queue
(B) Stack
(C) Sorted array
(D) Adjacency matrix
- 2) Edits A, B and C are pushed in that order onto an undo stack. Which edit does the first undo remove? (1)
(A) A
(B) B
(C) All three
(D) C
- 3) After edits A, B and C, two undo operations transfer actions to a redo stack. What is the redo stack from bottom to top? (1)
(A) C, B
(B) B, C
(C) A, B
(D) A, C
- 4) The redo stack contains C, B from bottom to top. Which edit is reapplied by one redo operation? (1)
(A) A
(B) C
(C) B
(D) Both B and C
- 5) In a conventional two-stack editor, what happens to the redo stack when a new edit is made after an undo? (1)
(A) It is reversed
(B) It is cleared
(C) It is unchanged
(D) It becomes the undo stack

Section B: Theory questions (25 marks)

- 6) Write a C program to read and display employee details including the date of joining using nested structure. Consider date of joining as nested structure. (2)
- 7) Assume that you are storing marks of students in 3 subjects. Write a C program that: i. Takes the number of students as input. ii. Dynamically allocates a 2D array using pointers. iii. Accepts marks for each subject. Calculates and prints the average marks of each student using only pointer arithmetic. (2)
- 8) Write a C program to dynamically allocate memory for 2D string arrays (array of strings) and perform concatenation using pointers. (2)
- 9) A music streaming service keeps track of songs in a playlist using a singly linked list. Each node stores the following details: SongID (integer), Title (string), Duration (in seconds). Write a C program to perform the following operations: Insert songs into the playlist in the order they are added. Remove all songs whose duration is less than a user-specified value (Use head pointer for performing this operation). Display the updated playlist. Include a main() function to demonstrate all functionalities. (5)
- 10) A mobile app named "MIT-BLR Connect" maintains contact details of alumni in three different modules: i) Add Recent Contacts Module: Displays recently interacted alumni in order of contact - new ones added at the beginning. ii) Traverse Module: Allows viewing alumni details in forward direction. iii) Develop a module to print the data in reverse order. Develop C functions considering singly linked list as the suitable data structure (5)
- 11) Write a function to convert an infix expression to postfix. Include the following functions as per the prototype. i) int InStackPrecedence(char op) ; to return the precedence of operator within the stack ii) int IncomingPrecedence(char op); to return the precedence of the incoming operator. iii) void Infix_Postfix(char infix[], char postfix[]); to convert given infix expression to postfix. Main function not required. Note: The infix expression may include ONLY the following operators (listed in the order of their precedence): ; ^ (exponentiation) → right associative ; * (multiplication), / (division) → left associative ; + (addition), - (subtraction) → left associative (5)
- 12) Write a C function bool isDuplicate(char s[]) that detects duplicate parentheses in a given expression using a stack. (2)
- 13) Using the following nested calls, justify why a stack is preferred over a queue in handling function calls. Explain the role of activation records (stack frames).
void fun1() { printf("Inside fun1\n"); }
void fun2() { fun1(); printf("Inside fun2\n"); }
int main() { fun2(); printf("Inside main\n"); } (2)

End of question paper